

INSTITUTO MAUÁ DE TECNOLOGIA



Lógica e Pensamento Computacional

Biblioteca Matplotlib

OBJETIVOS

- Revisar os conceitos das aulas anteriores
- Apresentar a biblioteca matplotlib
- Resolver os exercícios propostos

NumPy significa: **N**umerical **P**ython

É uma biblioteca criada para trabalhar com **cálculos numéricos e científicos** em Python.

Ela é muito utilizada em áreas como:

- Engenharia
- Física
- Estatística
- Ciência de dados
- Inteligência Artificial

O principal recurso do NumPy é o **array numérico**, tornando-se vantajosa em relação a biblioteca **math** quando são necessárias cálculos para vários valores.

O Python possui outros tipos de variáveis, cuja característica principal é armazenar uma “estrutura” de valores como se fosse uma única informação.

No curso trabalharemos duas delas:

- **Arrays:** Conjunto de valores numéricos que é utilizado junto com a biblioteca **numpy**.
- **Listas:** Conjunto de valores (numéricos ou não) que já está na configuração padrão do Python, sem a necessidade de importar bibliotecas



Aulas seguintes

Agora, vamos criar **arrays** para utilizar o **numpy**. Ao longo do curso veremos algumas formas de inicializar arrays. A primeira, e mais prática, é a inserção direta, ou seja, você digita os valores desejados no próprio código, utilizando o comando **array**:

```
import numpy as np  
  
x = np.array([0,3,5,-2,-4,3,8,-1])
```

Ou seja, esse comando cria uma variável x que é um conjunto numérico com os valores apresentados dentro dos colchetes e separados por vírgulas (a separação decimal é por ponto).

Assim, nesse caso, a variável x possui 8 valores armazenados na estrutura

Grande vantagem do **array**: Ao se realizar qualquer cálculo com uma variável **array**, ela realiza um *BROADCASTING* (Transmissão), para todos os valores dentro da estrutura:

```

▶ import numpy as np

x = np.array([0,3,5,-2,-4,3,8,-1])
y = 2*x
print(y)

```

_____ Cria-se o array

_____ Realiza-se uma operação com o array x resultando no array y

```

... [ 0  6 10 -4 -8  6 16 -2]

```

_____ O resultado são todos os elementos do array multiplicados por 2 neste caso.

Outras formas de criar arrays são com comandos que criam sequências numéricas pré-defindas:

np.arange (a,b,n) -> Cria um array cujo ponto inicial é “a” e tem como limite o valor “b”. A subdivisão é dada pelo passo “n”

np.linspace (a,b,n) -> Cria um array com início no ponto “a” e término no ponto “b”. “n” será a quantidade de pontos

np.zeros (n) -> Cria um array com n valores (todos iguais a zero)

np.ones (n) -> Cria um array com n valores (todos iguais a um)

Matplotlib: Biblioteca do Python utilizada para **visualização de dados**. Ela permite criar gráficos de forma simples e rápida, sendo muito utilizada em:

- Engenharia
- Ciência de dados
- Física
- Economia
- Análise experimental

Com ela podemos transformar **valores numéricos em gráficos**, facilitando a interpretação dos resultados.

Importação: `import matplotlib.pyplot as plt` → “Apelido” conhecido

O **Matplotlib** possui uma grande variedade de tipos de gráficos, destacando-se:

- Linhas
 - Dispersão
- } Abordaremos no curso

- Barra
 - Histograma
 - Pizza
 - Área
 - Contornos
 - Campos de Vetores
 - Superfícies
- } Conhecimento que será adquirido nas próximas disciplinas, considerando que você já conhecerá a biblioteca!

Gráfico de dispersão: Também conhecido como gráfico de pontos, utiliza um plano cartesiano (eixos x e y) e através de pares ordenados $[(x_1, y_1), (x_2, y_2), \dots]$, posiciona-se esses pontos no gráfico:

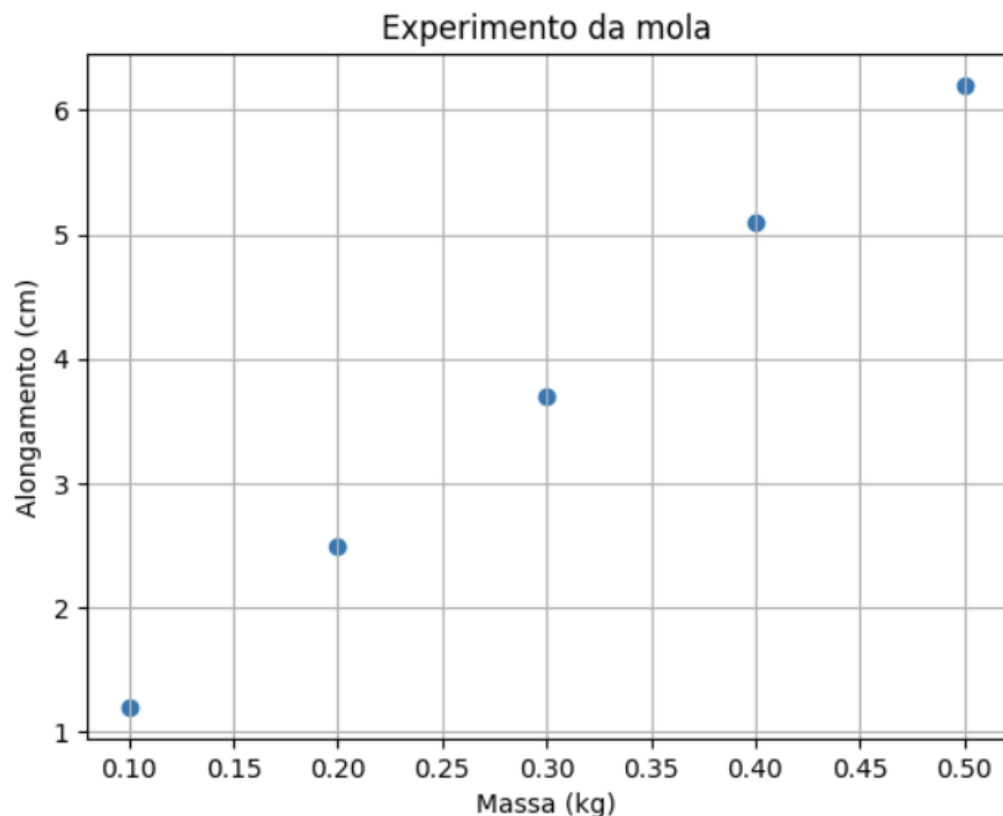


Gráfico de uma mola linear posicionada na vertical, onde dada uma massa presa a ela, verifica-se qual o alongamento produzido

Para montarmos este gráfico, precisamos das coordenadas x e y organizadas em variáveis estruturadas (array ou lista). Assim, para o caso de arrays, o uso do numpy junto com o matplotlib é muito comum.

Após isso, gera-se o gráfico com o comando “**plt.scatter()**”:

```
▶ import numpy as np  
import matplotlib.pyplot as plt
```

_____ → Importação das bibliotecas

```
massa = np.linspace(0.1,0.5,5)  
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])
```

_____ → Arrays criados

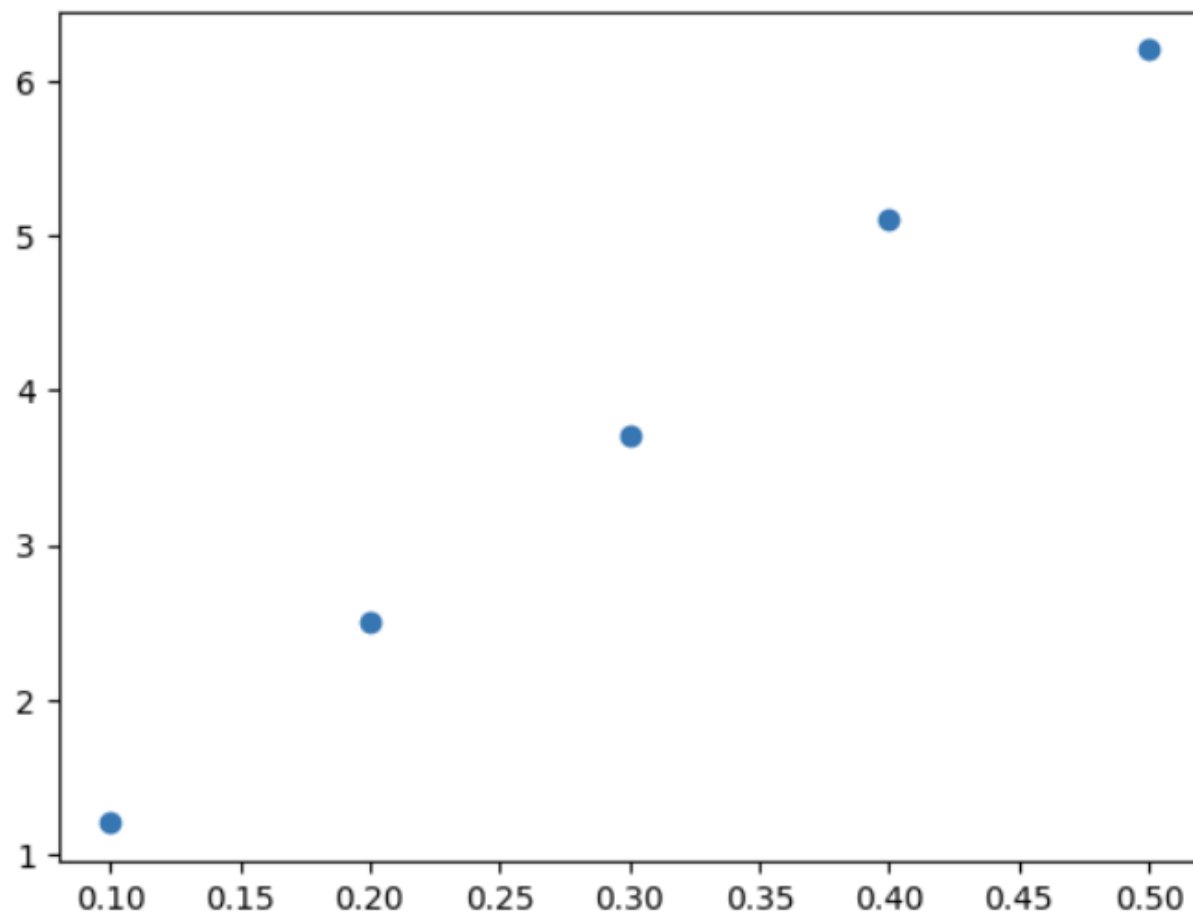
```
plt.scatter(massa, alongamento)
```

_____ → Comando para gerar o gráfico

```
plt.show()
```

_____ → Comando para mostrar o gráfico na tela

O resultado será este gráfico:



Este gráfico não está com a “formatação” do gráfico anterior. O **matplotlib** tem comandos para realizar isso.

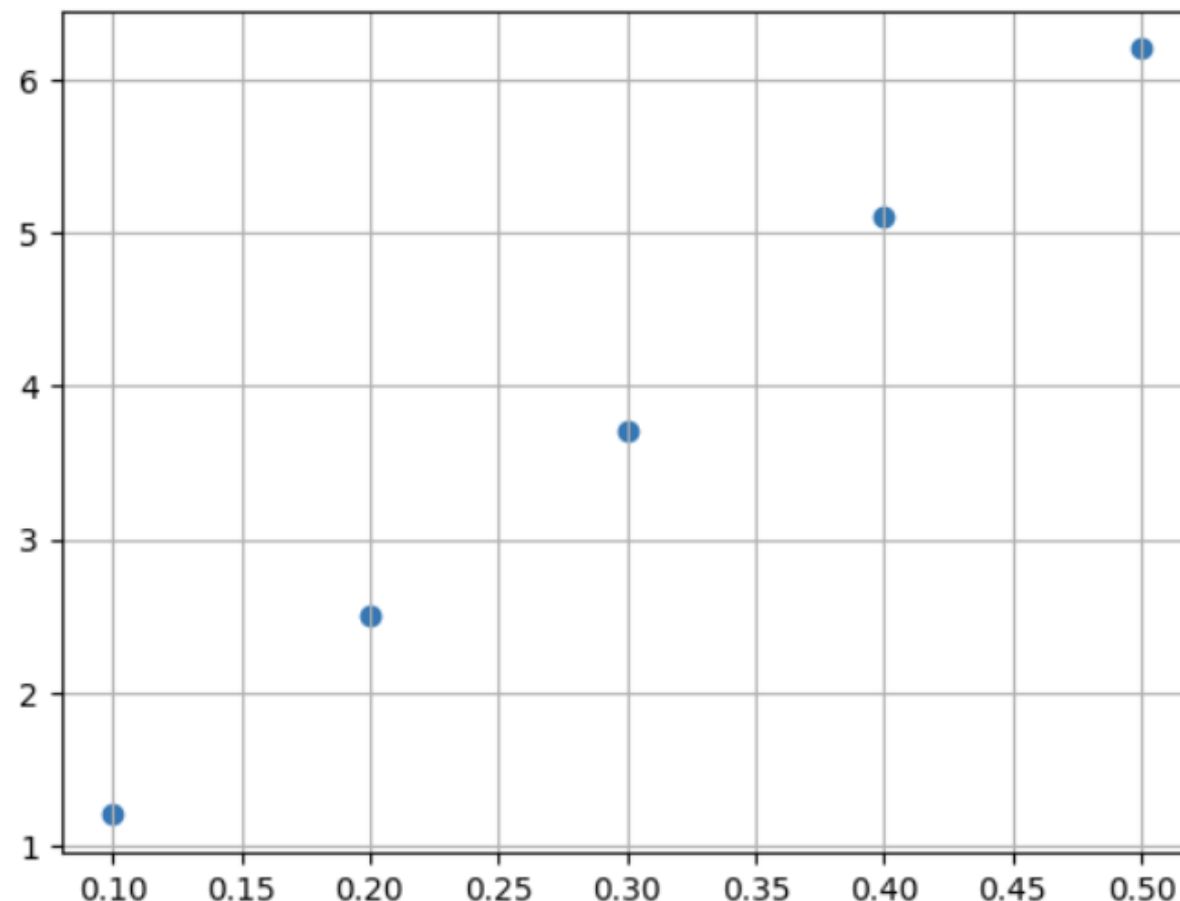
Comandos de edição:

- **plt.grid()** : Coloca linhas de grade

```
▶ import numpy as np  
import matplotlib.pyplot as plt
```

```
massa = np.linspace(0.1,0.5,5)  
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])
```

```
plt.scatter(massa, alongamento)  
plt.grid()  
plt.show()
```



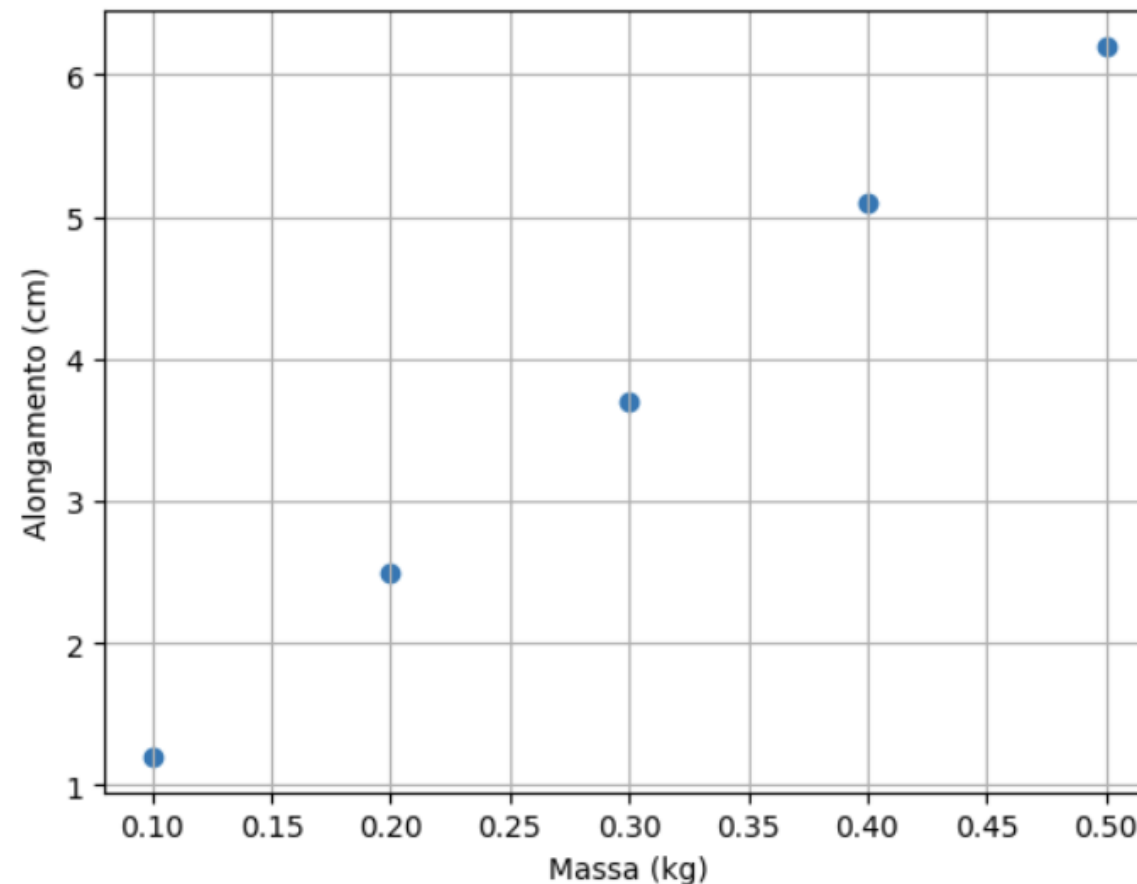
Comandos de edição:

- **plt.xlabel()** e **plt.ylabel()** : Títulos nos eixos

```
▶ import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento)
plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.grid()
plt.show()
```



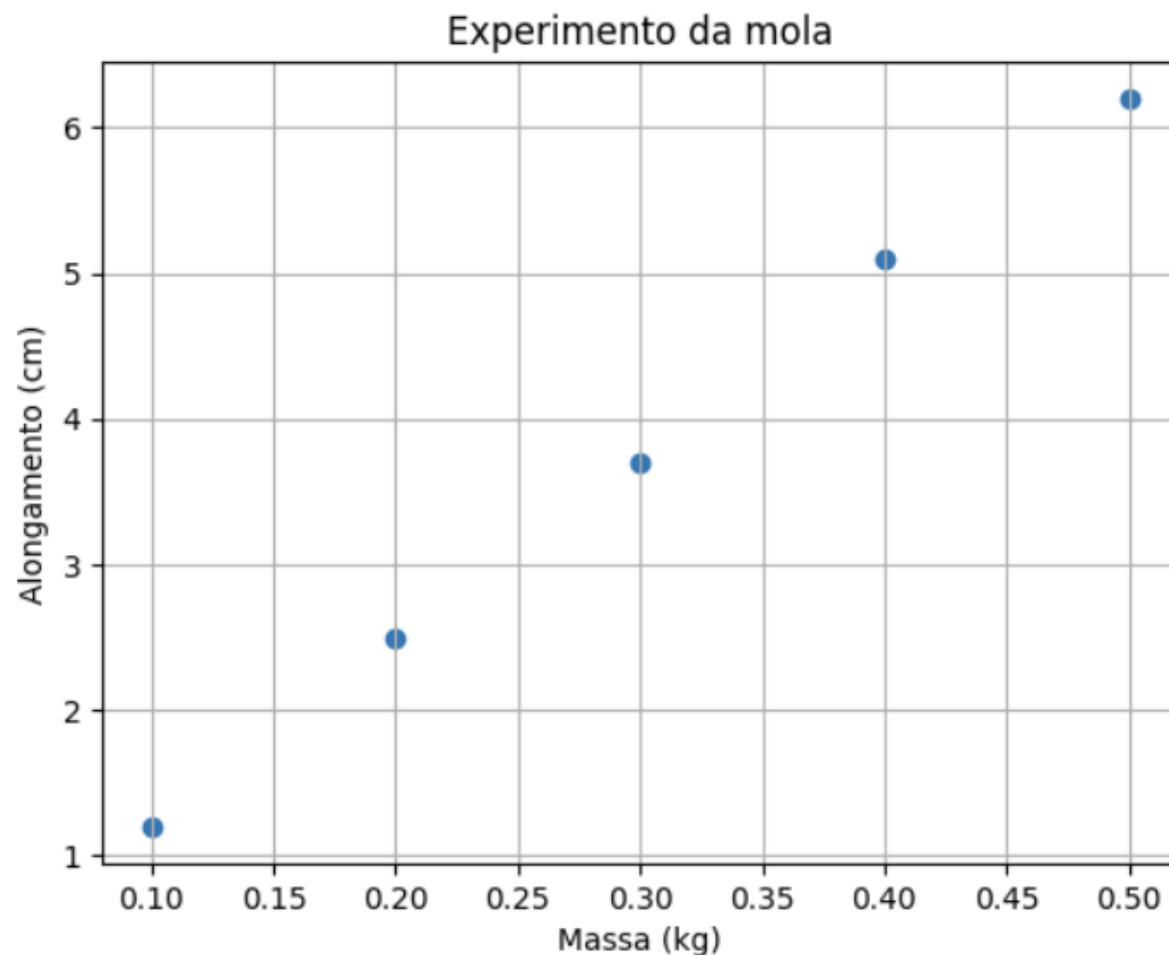
Comandos de edição:

- **plt.title()** : Título do gráfico

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento)
plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



Comandos de edição:

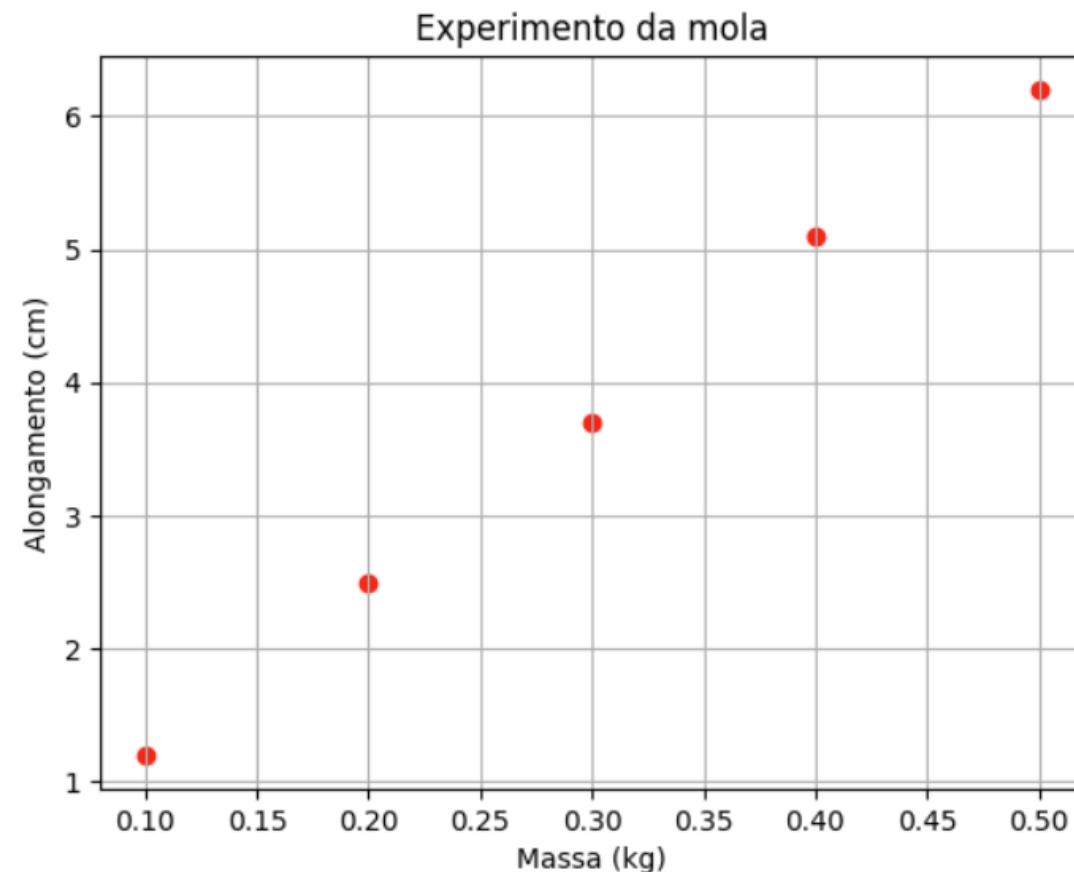
- **Cor dos pontos:** Diretamente no comando `plt.scatter()`:

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento, color='red')
plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```

Instrução logo
após a inserção
dos pontos x e y



Comandos de edição:

- **Cor dos pontos:** Diretamente no comando `plt.scatter()`
- Lista de cores mais utilizadas:

Cor	Código
Azul	'blue'
Vermelho	'red'
Verde	'green'
Preto	'black'
Branco	'white'
Amarelo	'yellow'
Laranja	'orange'
Roxo	'purple'
Rosa	'pink'
Marrom	'brown'
Cinza	'gray'

Comandos de edição:

- **Tamanho dos pontos:** Diretamente no comando `plt.scatter()`:

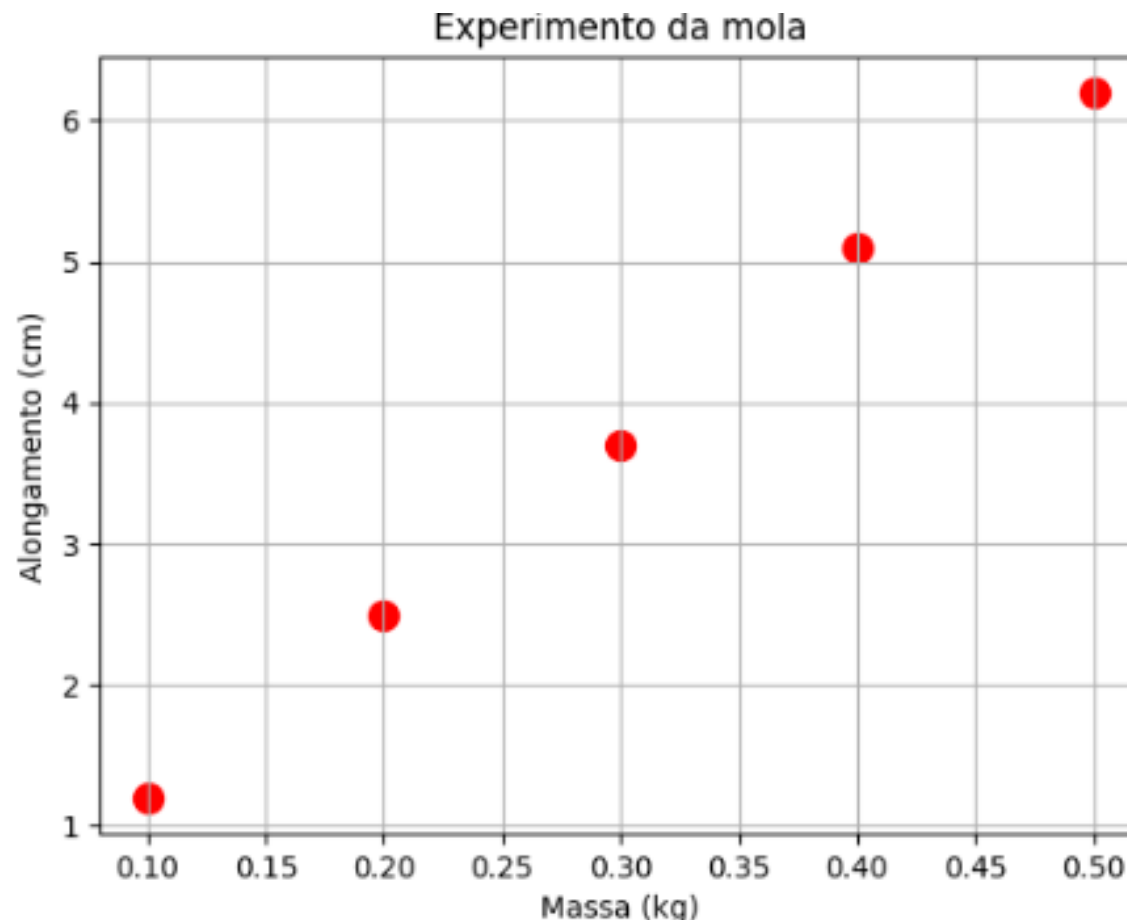
```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento, color='red', s=100)

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```

Instrução usando
o comando “s=”
(size)



Comandos de edição:

- **Formato dos pontos:** Diretamente no comando `plt.scatter()`:

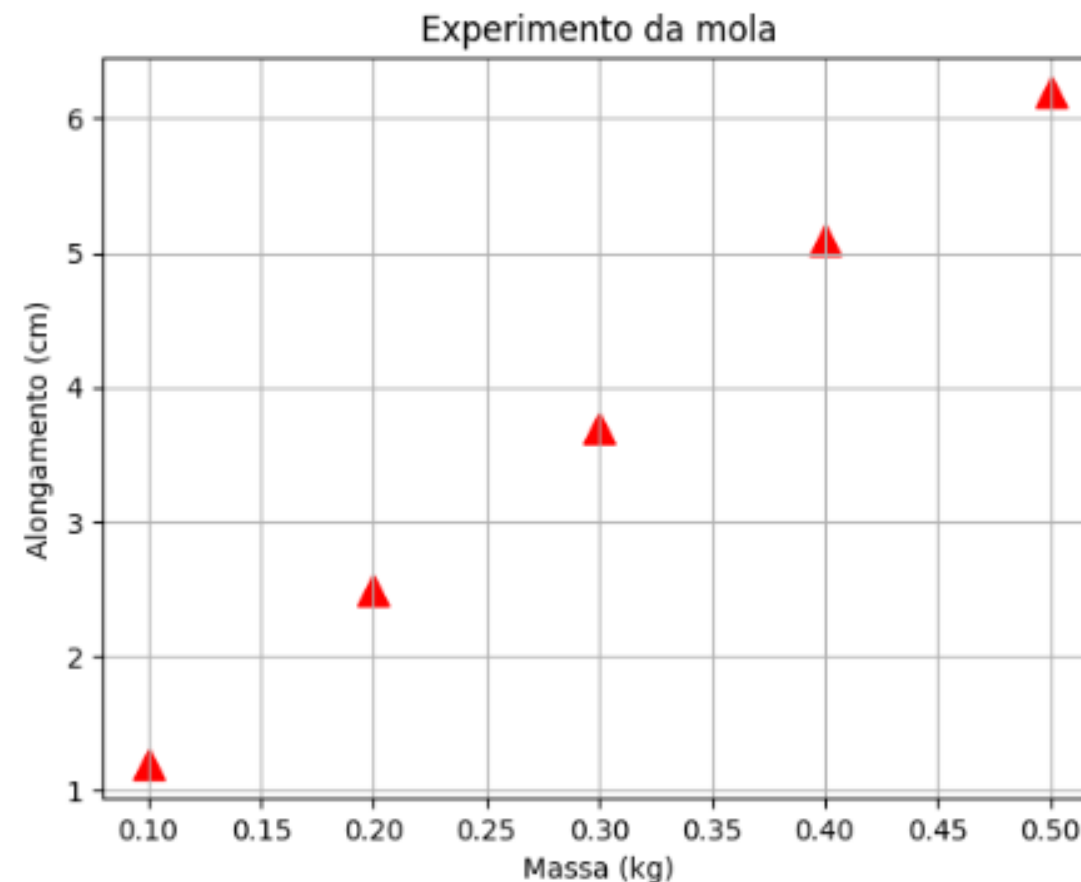
```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.scatter(massa, alongamento,color='red',s=120,marker='^')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```

Instrução usando o comando “marker”



Comandos de edição:

- **Formato dos pontos:** Diretamente no comando `plt.scatter()`

Marker	Forma
'o'	círculo
's'	quadrado
'^'	triângulo para cima
'v'	triângulo para baixo
'<'	triângulo esquerda
'>'	triângulo direita
'D'	losango
'p'	pentágono
'*'	estrela
'+'	mais
'x'	X
'.'	ponto pequeno

- Lista de formatos mais utilizados:

Gráfico de linha: Utiliza um plano cartesiano (eixos x e y) e através de pares ordenados $[(x_1, y_1), (x_2, y_2), \dots]$, liga os pontos através de segmentos de reta.

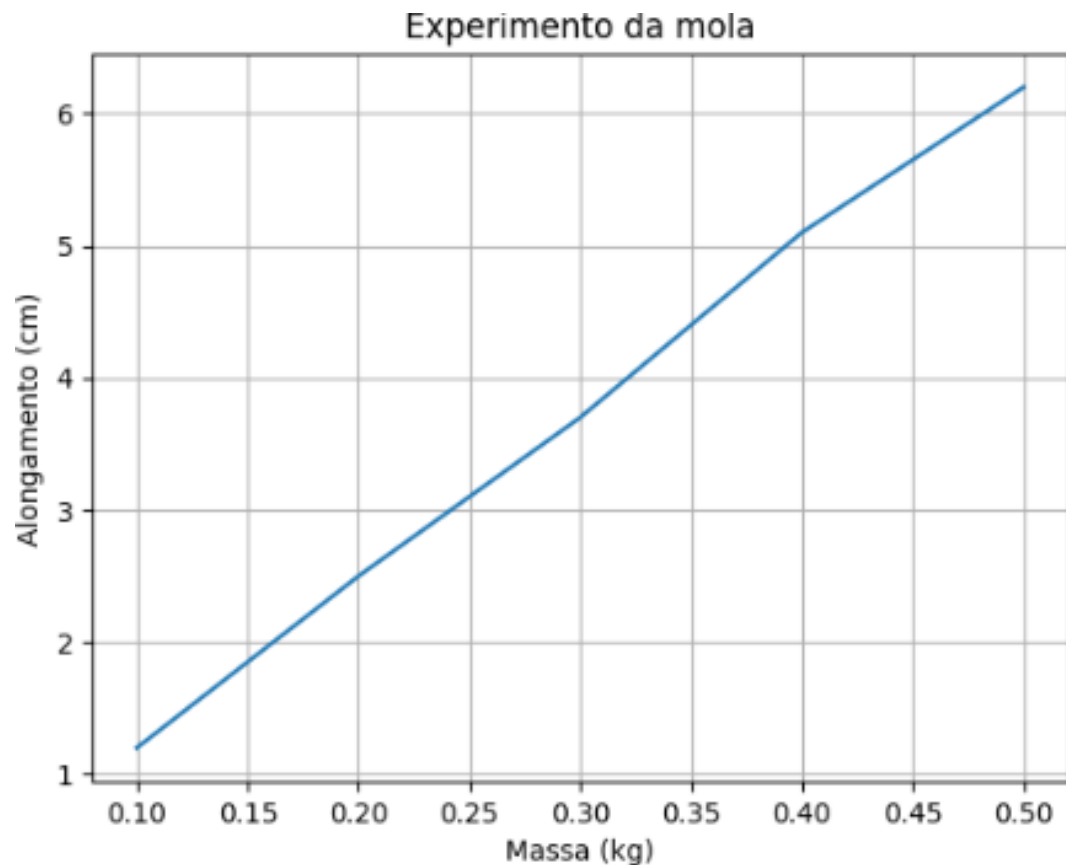


Gráfico de uma mola linear posicionada na vertical, onde dada uma massa presa a ela, verifica-se qual o alongamento produzido. Neste caso, os pontos foram ligados por segmentos de reta

Para montarmos este gráfico, precisamos das coordenadas x e y organizadas em variáveis estruturadas (array ou lista). Assim, para o caso de arrays, o uso do numpy junto com o matplotlib é muito comum.

Após isso, gera-se o gráfico com o comando “**plt.plot()**”:

```
import numpy as np
```

```
import matplotlib.pyplot as plt
```

Importação das bibliotecas

```
massa = np.linspace(0.1,0.5,5)
```

```
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])
```

Arrays criados

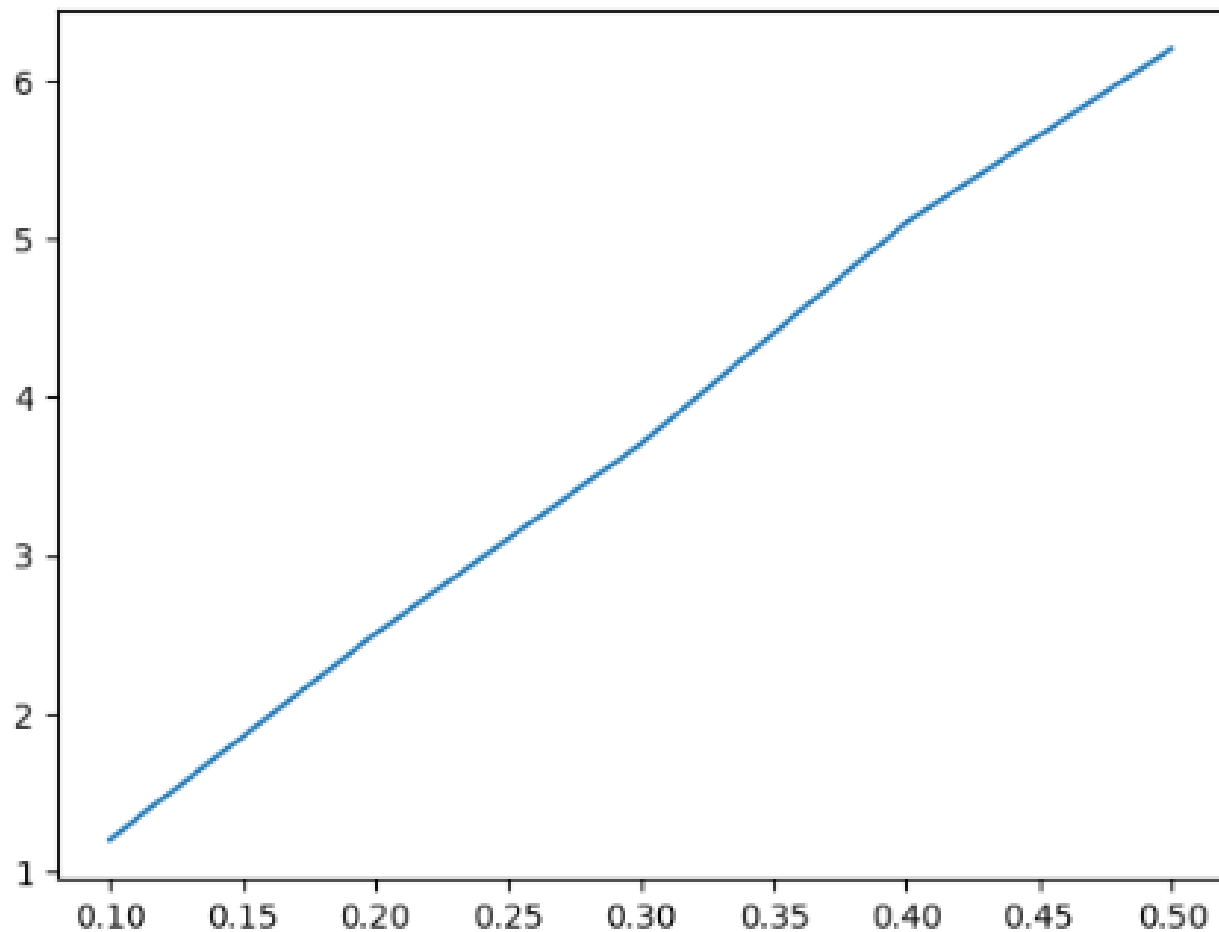
```
plt.plot(massa, alongamento)
```

Comando para gerar o gráfico

```
plt.show()
```

Comando para mostrar o gráfico na tela

O resultado será este gráfico:



Este gráfico não está com a “formatação” do gráfico anterior. O **matplotlib** tem comandos para realizar isso.

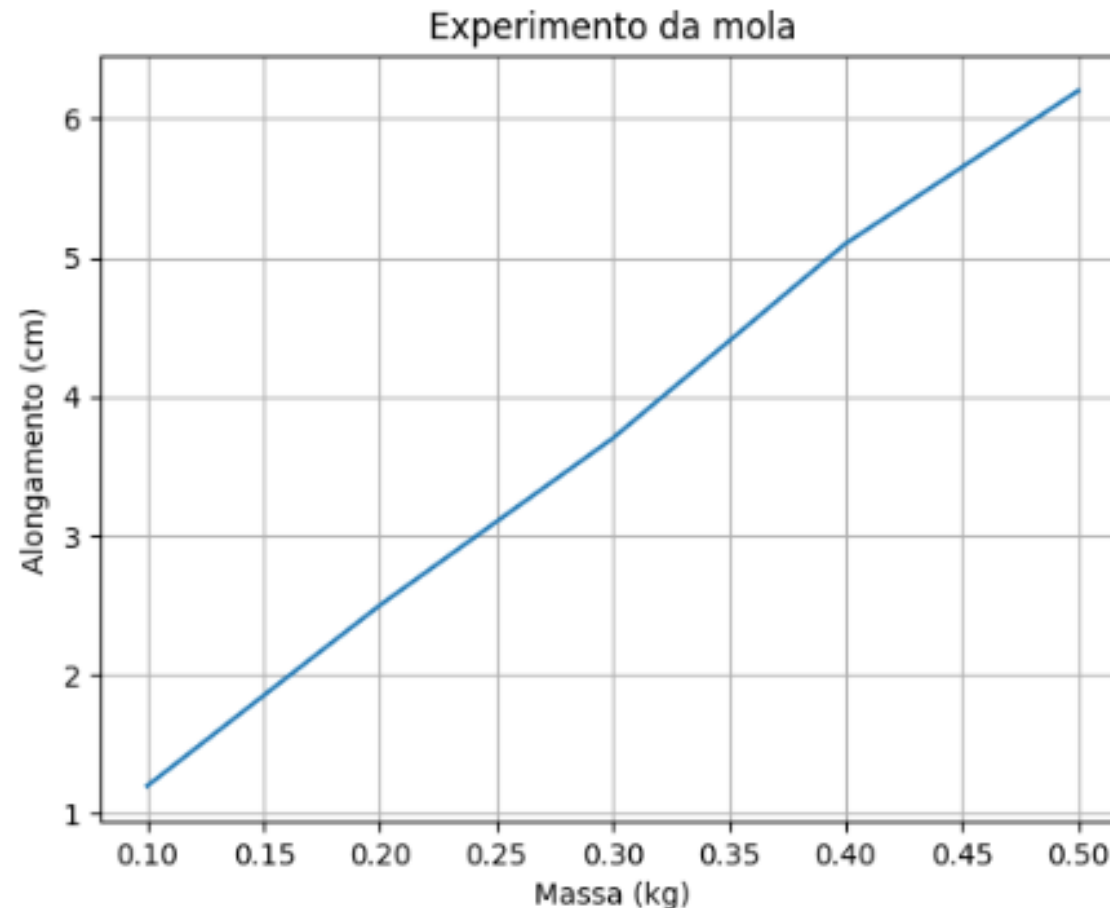
Os comandos externos ao **plt.plot()** e o **plt.scatter()** são os mesmos:

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento)

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



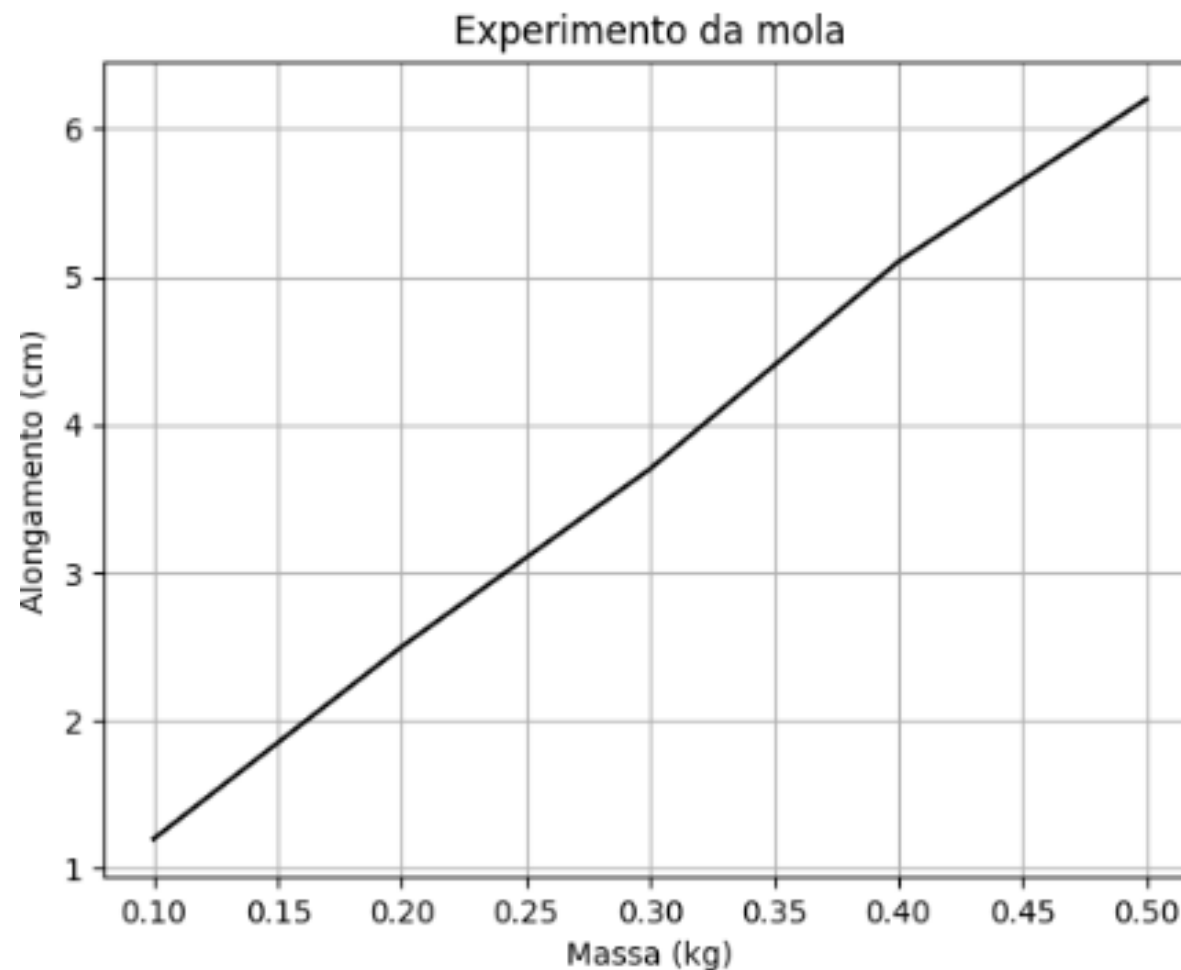
A configurações de cores são as mesmas:

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento, color = 'black')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



No **plt.plot()** podemos configurar espessura e estilo de linha. Isso é colocado diretamente dentro dos parênteses do plot:

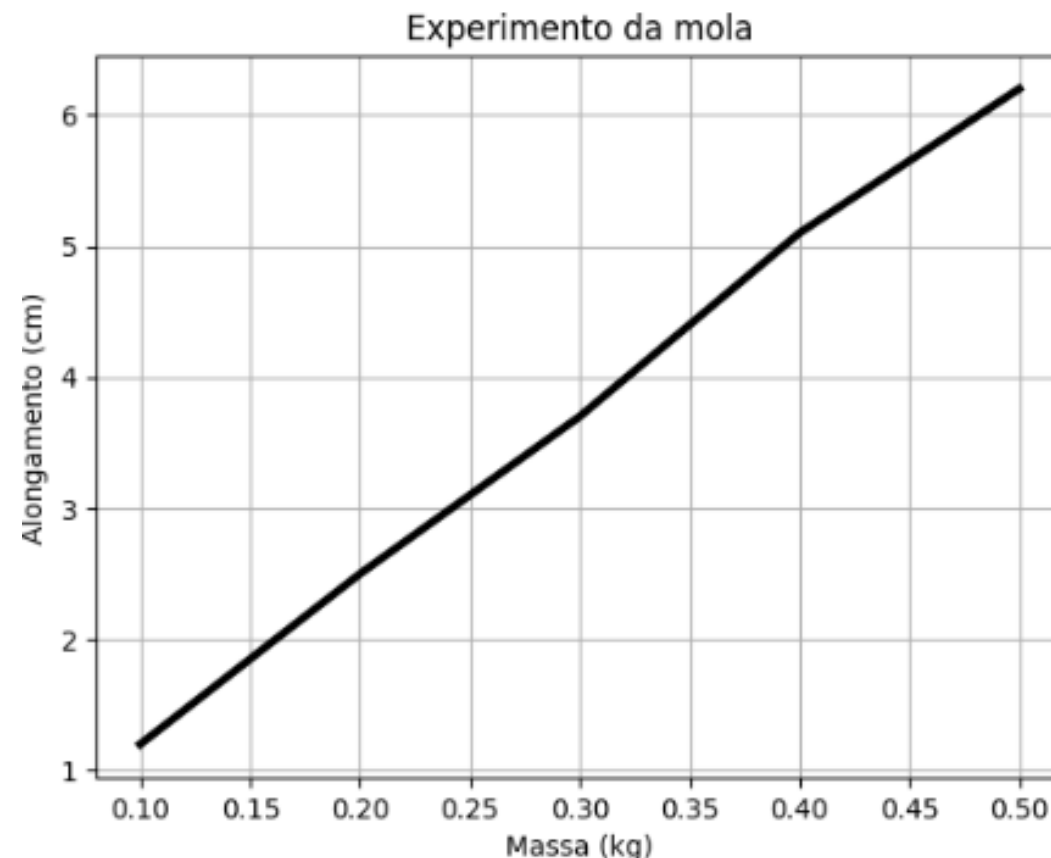
- Na espessura podemos utilizar “linewidth = “ ou simplesmente “lw = “

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento, color='black', lw=3)

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



No **plt.plot()** podemos configurar espessura e estilo de linha. Isso é colocado diretamente dentro dos parênteses do plot:

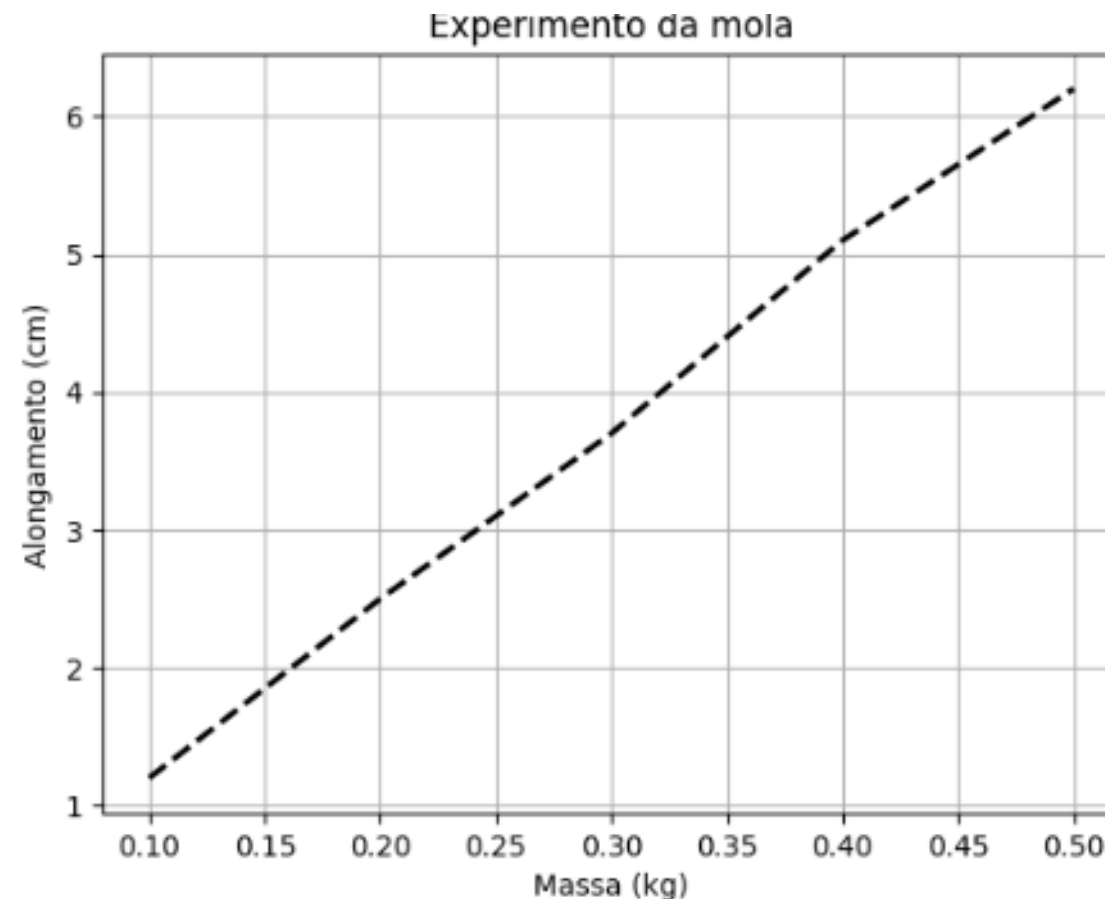
- No estilo podemos utilizar “`linestyle =`” ou simplesmente “`ls =`”

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento,color='black',lw=2,ls='--')

plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.grid()
plt.show()
```



No **plt.plot()** podemos configurar espessura e estilo de linha. Isso é colocado diretamente dentro dos parênteses do plot:

- No estilo podemos utilizar “`linestyle =` “ ou simplesmente “`ls =` “
- Estilos mais comuns:

Código

Tipo de linha

' - '

linha contínua

' - - '

linha tracejada

' - . '

traço-ponto

' : '

pontilhada

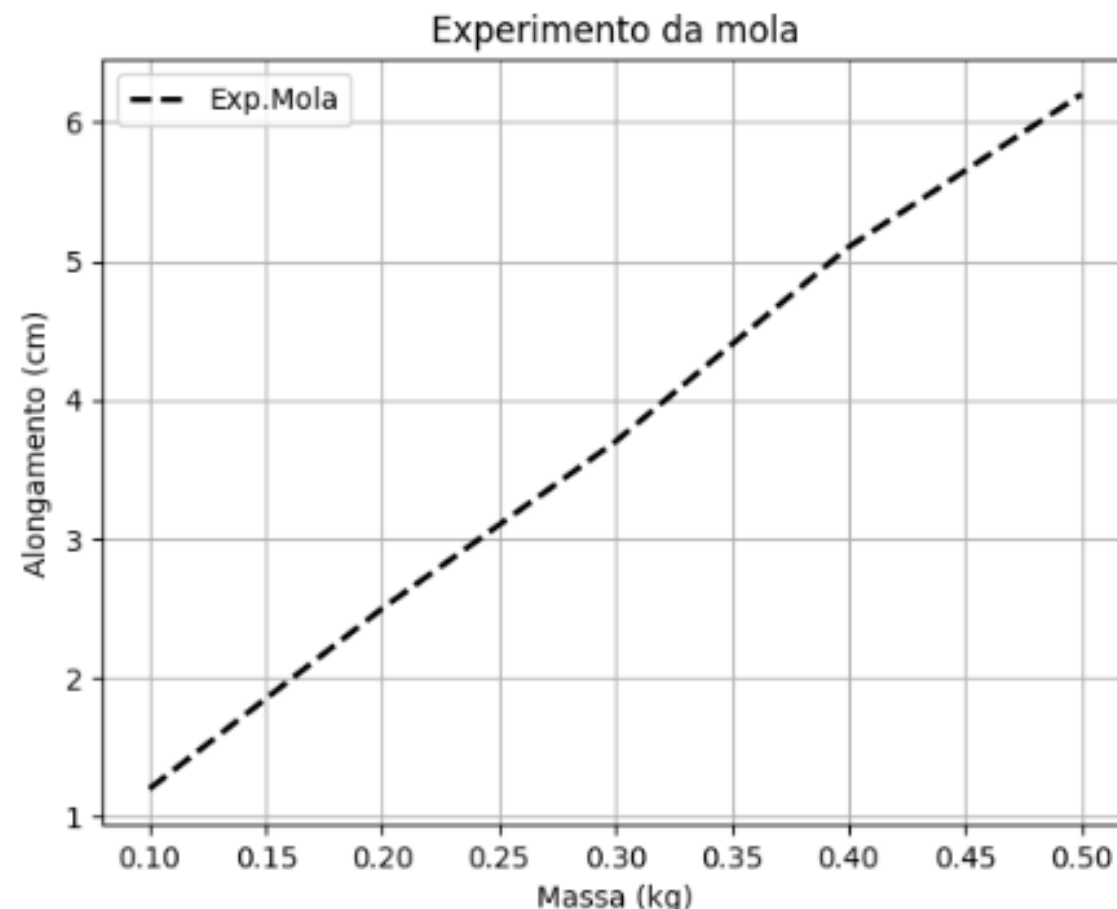
Legenda: Tanto o **plt.scatter()** como o **plt.plot()** podem ter legendas, basta utilizar o comando **plt.legend()**. Para determinar o nome que estará na legenda, deve-se utilizar o comando “**label=**” dentro scatter ou plot:

```
import numpy as np
import matplotlib.pyplot as plt

massa = np.linspace(0.1,0.5,5)
alongamento = np.array([1.2,2.5,3.7,5.1,6.2])

plt.plot(massa, alongamento,color='black',lw=2,ls='--',label="Exp.Mola")

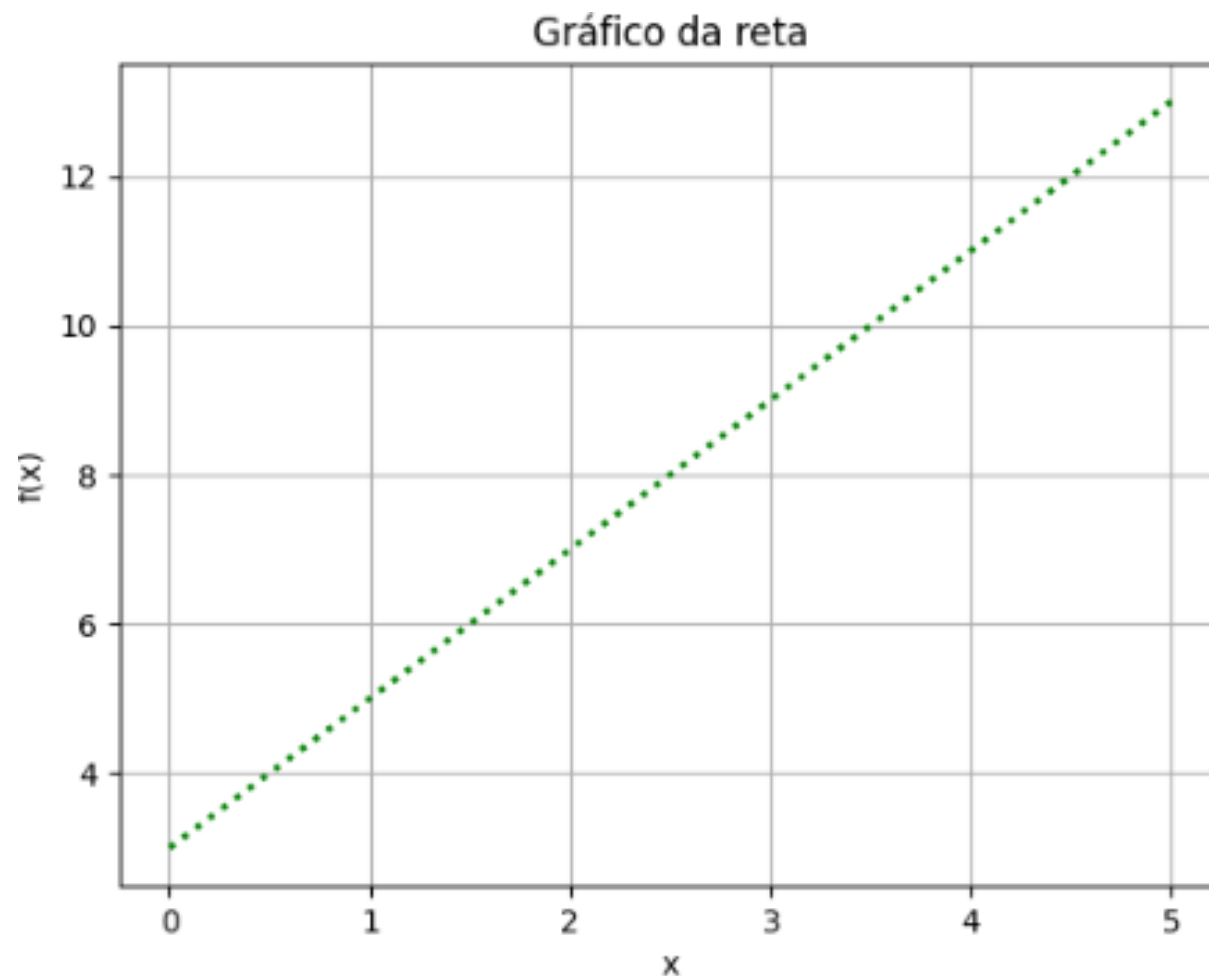
plt.xlabel("Massa (kg)")
plt.ylabel("Alongamento (cm)")
plt.title("Experimento da mola")
plt.legend()
plt.grid()
plt.show()
```



Exemplo 1: Utilize a sub-rotina `reta` criada anteriormente e crie um array `x` de 0 a 5 com 6 pontos. Calcule a função para todos esses pontos e faça um gráfico com a reta na cor verde, espessura de 2 mm e pontilhada. Invente um título para o gráfico, nomeie os eixos e coloque linhas de grade.

```
# _____  
#Bibliotecas  
import numpy as np  
import matplotlib.pyplot as plt  
  
# _____  
#Subrotinas  
def reta(x):  
    fx = 2*x+3  
    return fx  
  
# _____  
#Programa Principal  
x = np.linspace(0,5,6)  
y = reta(x)  
  
plt.plot(x,y,color='green',lw = 2, ls = ':')  
plt.title("Gráfico da reta")  
plt.xlabel("x")  
plt.ylabel("f(x)")  
plt.grid()  
plt.show()
```

Exemplo 1: Utilize a sub-rotina `reta` criada anteriormente e crie um array `x` de 0 a 5 com 5 pontos. Calcule a função para todos esses pontos e faça um gráfico com a reta na cor verde, espessura de 2 mm e pontilhada. Invente um título para o gráfico, nomeie os eixos e coloque linhas de grade.



Exemplo 2: Agora troque a função `reta(x)` por uma função chamada “`parab(x)`” que calcula a parábola $f(x) = -x^2 + 5x - 6$. Mude a cor da linha para preto e retire o pontilhado. Além disso, altere título do gráfico e nomes dos eixos para manter a coerência.

```
# _____
#Bibliotecas
import numpy as np
import matplotlib.pyplot as plt

# _____
#Subrotinas
def parab(x):
    fx = -x**2+5*x-6
    return fx

# _____
#Programa Principal
x = np.linspace(0,5,6)
y = parab(x)

plt.plot(x,y,color='black',lw = 2)
plt.title("Gráfico da parábola")
plt.xlabel("x")
plt.ylabel("f(x)")
plt.grid()
plt.show()
```


Exemplo 2: Agora troque a função `reta(x)` por uma função chamada “`parab(x)`” que calcula a parábola $f(x) = -x^2 + 5x - 6$. Mude a cor da linha para preto e retire o pontilhado. Além disso, altere título do gráfico e nomes dos eixos para manter a coerência.

O desenho da parábola não parece correto, como resolver?



Aumentando o linspace para 10 pontos:



Aumentando o linspace para 50 pontos:

